



Deep Learning

王 海 林

计算机与人工智能学院



01

历史发展

02

知识基础

03

模型训练

04

相关思考



01

历史发展

02

知识基础

03

模型训练

04

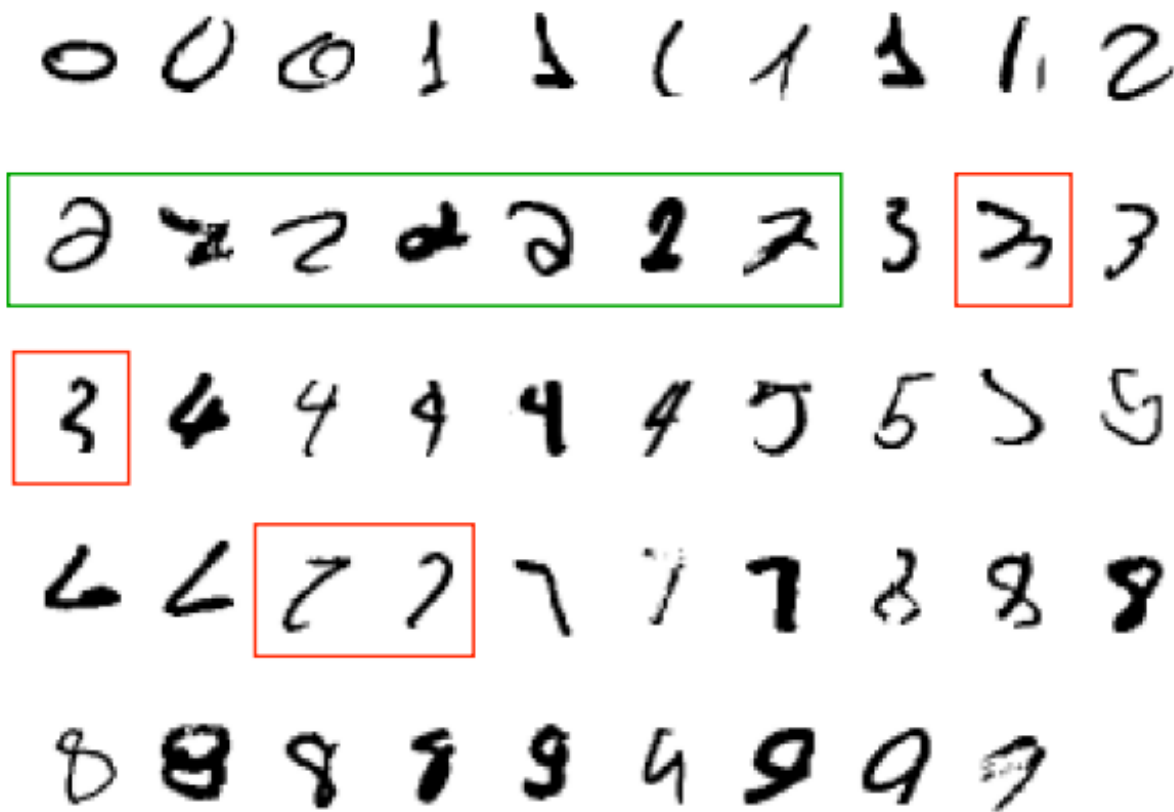
相关思考



01

历史发展

如何识别“2”

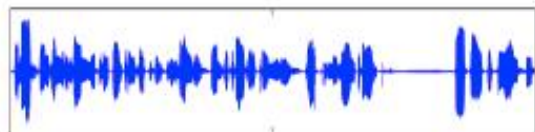


每个数字对应一个 28×28 像素的图像，可以表示为一个由 784 个实数组成的向量 x 。

目标是建立一个机器，能够以这样的向量 x 作为输入，以数字 0 到 9 为输出。

MNIST 数据集 <http://yann.lecun.com/exdb/mnist/>

语音识别



讯飞听见

微软小冰

苹果Siri

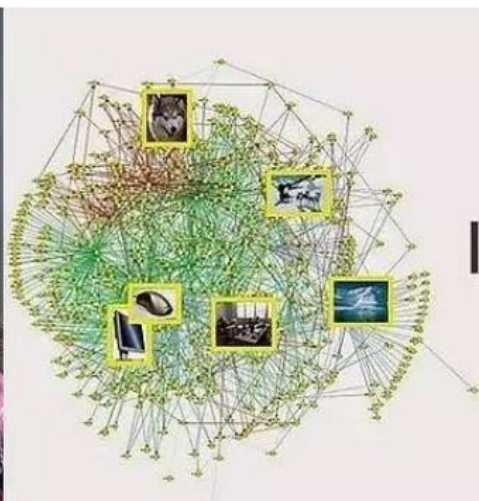
01

历史发展

人脸识别



Example training images
for each orientation



IMAGENET

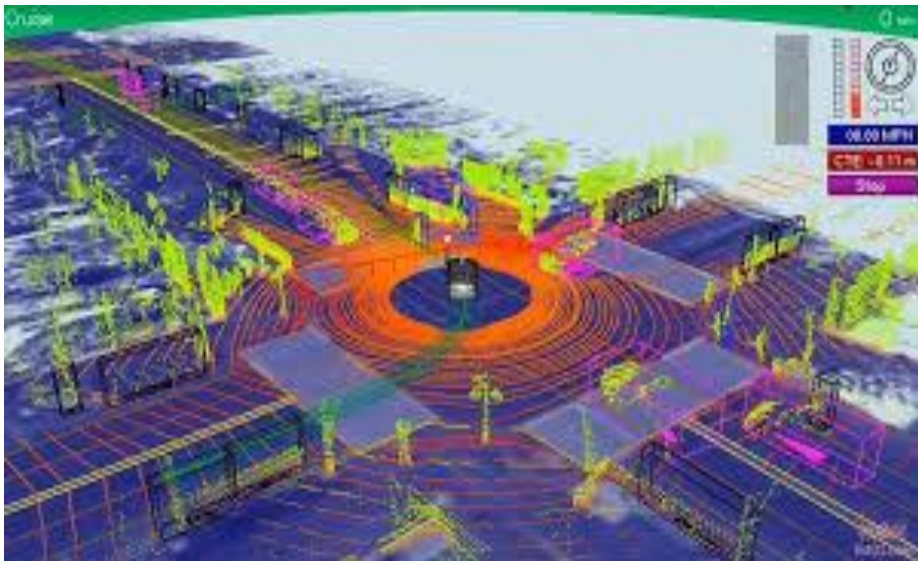
[illegible]

01

历史发展

股票市场预测





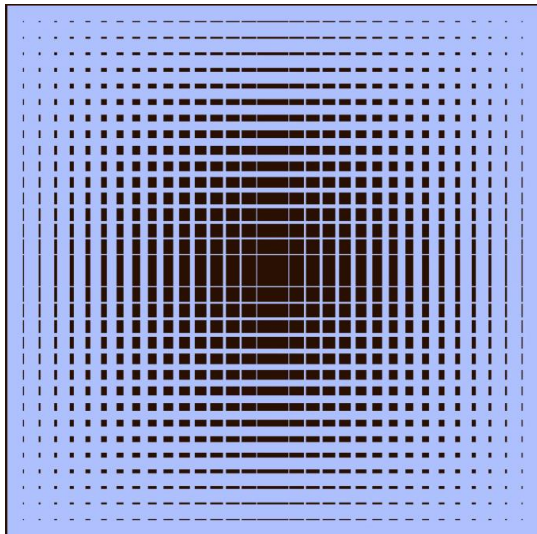
自动驾驶



01

历史发展





Dall·E

ChatGPT

创建 图像 最低
带有 AI 的字词

描述要创建的内容

 加入和创建

图像创建器基于文本生成 AI 图像。 [了解详细信息。](#)

你将收到有关 Microsoft Rewards 的电子邮件，其中包括有关 Microsoft 和合作伙伴产品的优惠。你还将收到有关必应图像创建器的通知。继续操作即表示你同意下面的 Rewards 条款和图像创建器条款。

[隐私](#) | [Rewards 条款](#) | [图像创建器条款](#)

 在一个位置进行搜索、聊天和创作。
[试用新必应中的图像创建器。](#)



What is Deep Learning?

Artificial Intelligence



any technique that enables computers to mimic human behavior

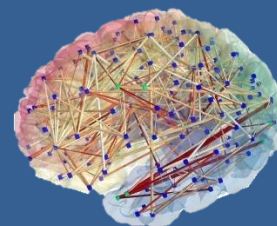
Machine Learning

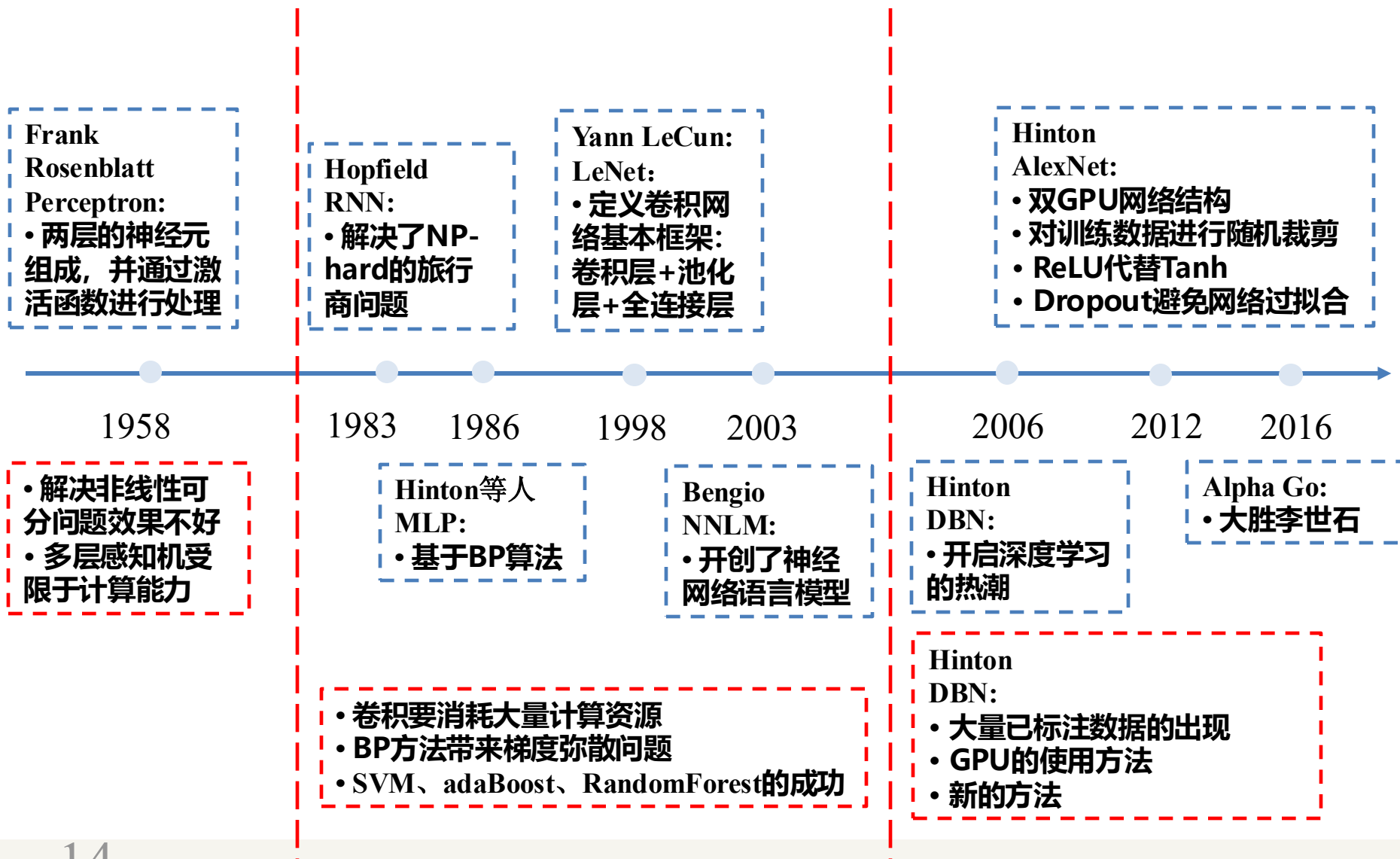


Ability to learn without explicitly being programmed

Deep Learning

Extract patterns from data using neural networks

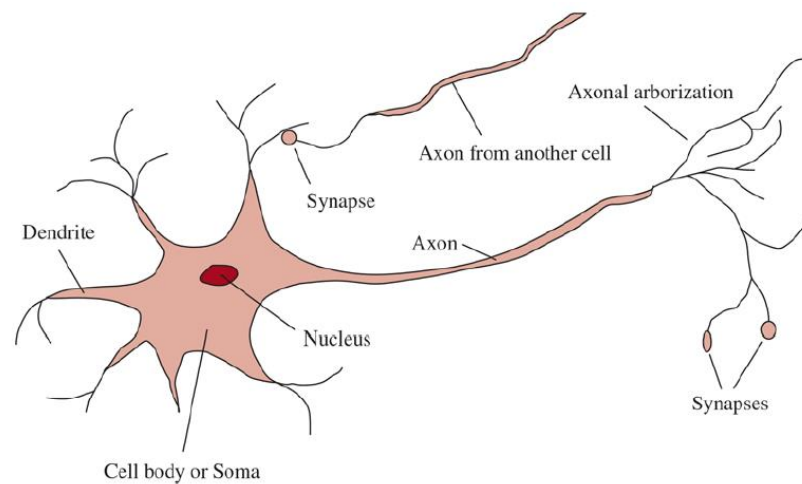
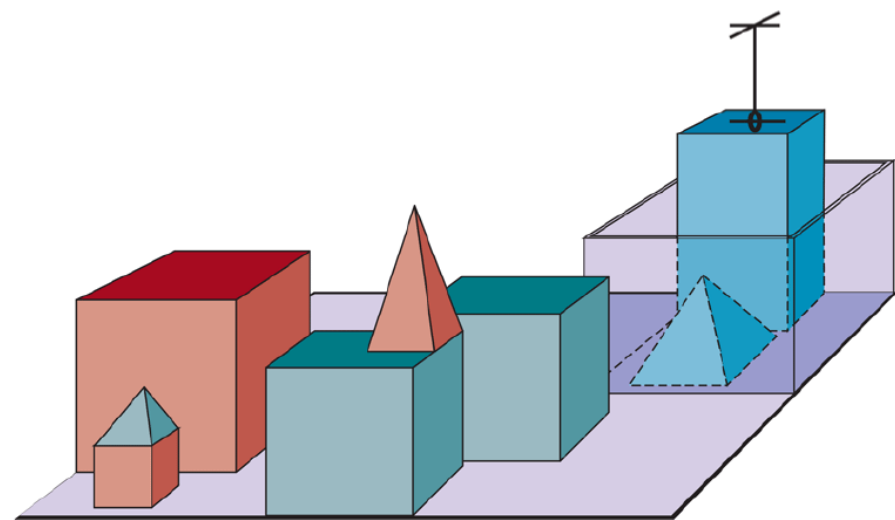




为什么要研究深度学习？ 因为需要高效建模。

Question 1

人脑如何完成各种任务（建模）？

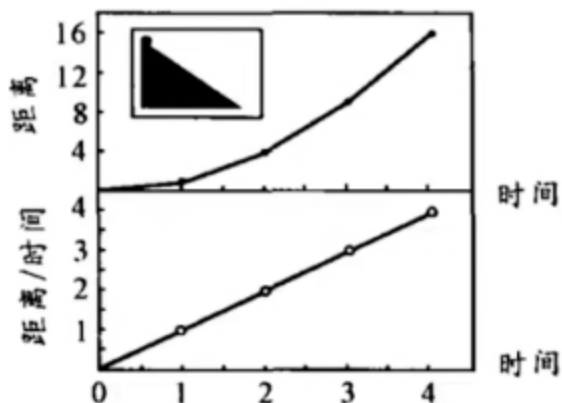


人脑通过神经元各种**连接与突触**的刺激可以判别图中不同物体

Question 2

现实世界如何模拟人脑建模？

现实世界：模型驱动



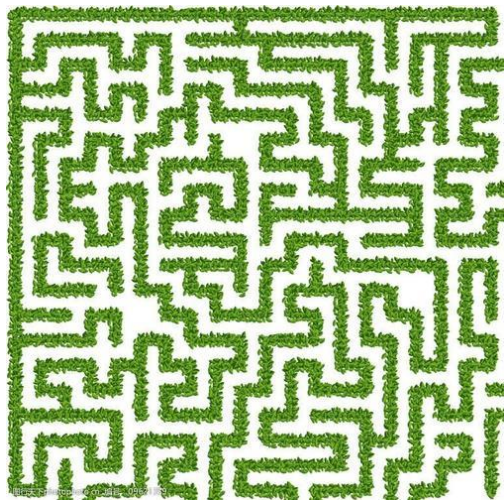
伽利略对落体运动的变化，提出合理的假设，认为速度与时间存在着简单的比例关系，也即 v/t 为常数(公式得到验证)

$$S=(1/2)at^2$$

1. 观察世界（自由落体）
2. 建立模型（加速度公式）
3. 验证模型（实验验证）

Question 3

现实世界更加复杂时如何建模？



迷宫

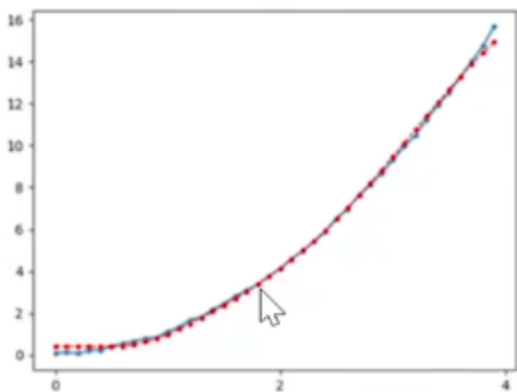


股票趋势

Question 4

深度学习如何模拟人脑建模？

深度学习：数据驱动



神经网络对数据进行拟合

```
(hidden1):Linear(in_features=1,out_features=20)  
(hidden2):Linear(in_features=20,out_features=20)  
(predict):Linear(in_features=20,out_features=1)
```

以自由落体为例：模型并不知道 $S = (1/2)at^2$ ，但是可以通过数据去拟合实际情况

最终得到包含一套参数的映射。



01

历史发展

深度学习~寻找一个映射 →

- Computer Vision



- Natural language processing

中文情感分析 核酸检测时间太久了



BaiduSenta: 负向(Negative)_负向概率:0.8852
SnowNLP: 负向(Negative)_负向概率:0.6733

- Reinforcement Learning



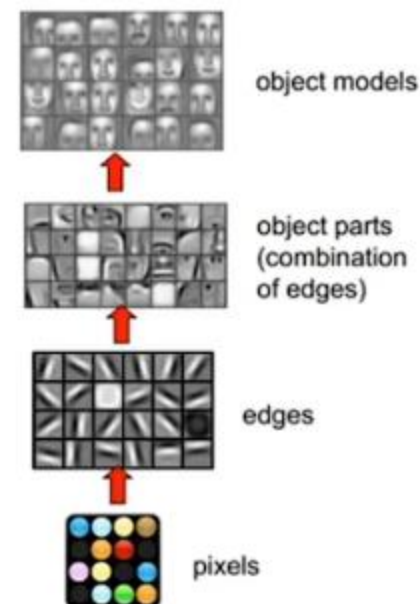
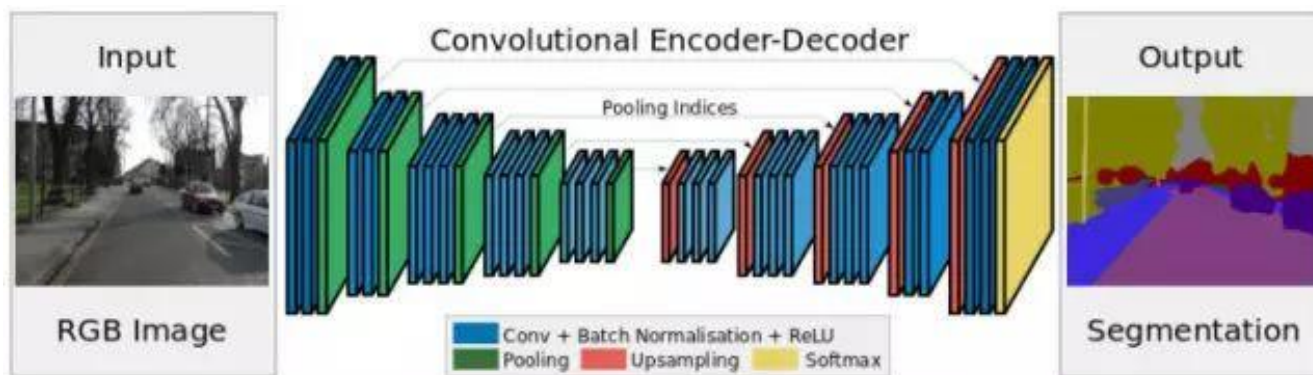
上下左右

- Recommendation System



Question 5

什么叫深度学习？为什么不用机器学习？



- 直接从数据学习特征，层数越多，表达能力越强
- 深度学习自动从数据中学习特征，机器学习需要手工设计特征



01

历史发展

02

知识基础

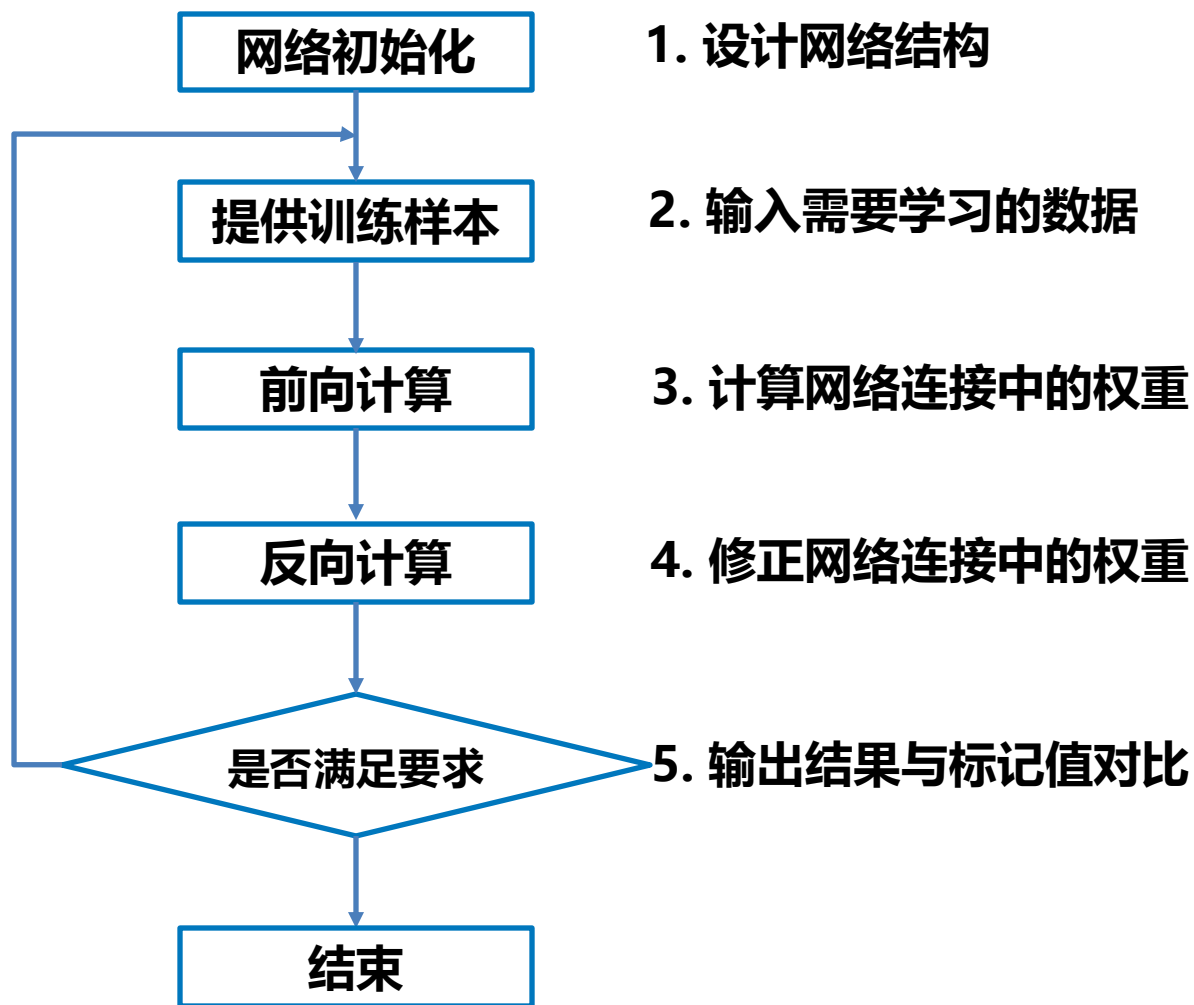
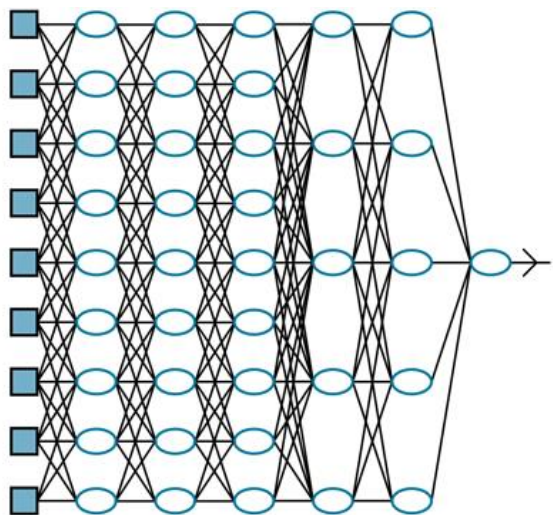
03

模型训练

04

相关思考

如何从数据中学习映射？



学习方式区分



标注数据

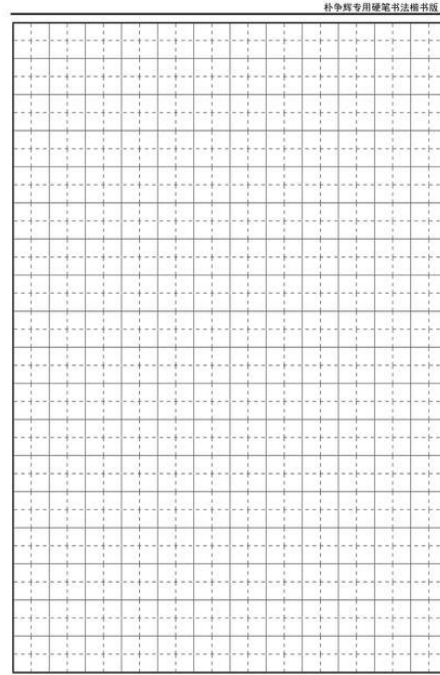
有监督学习
临摹书法



标注

未标注

半监督学习
临摹书法后自由发挥

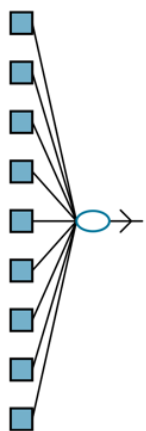
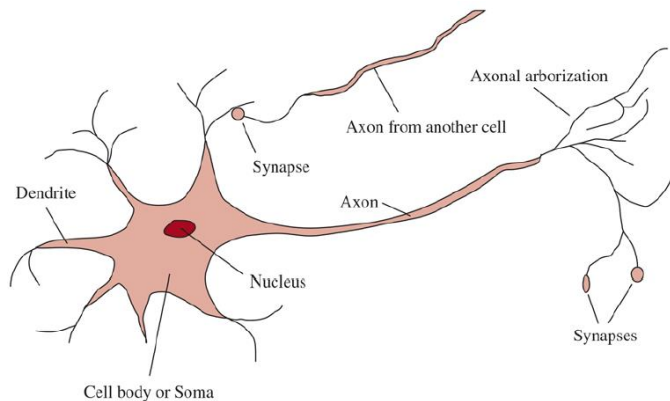


未标注数据

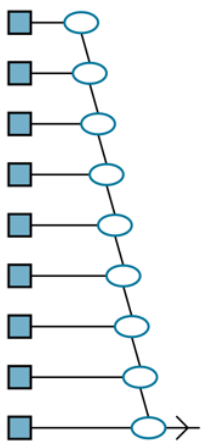
无监督学习
自行创作

例子

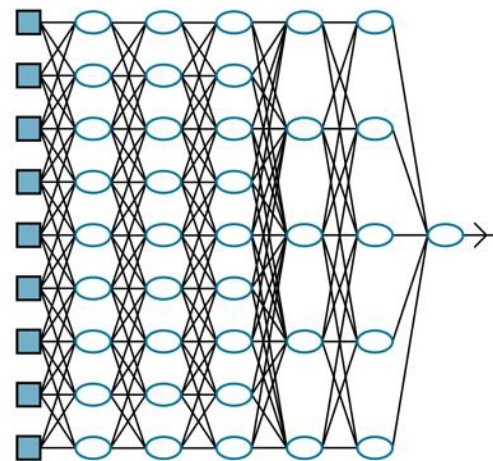
神经元-神经网络基本单元



神经元

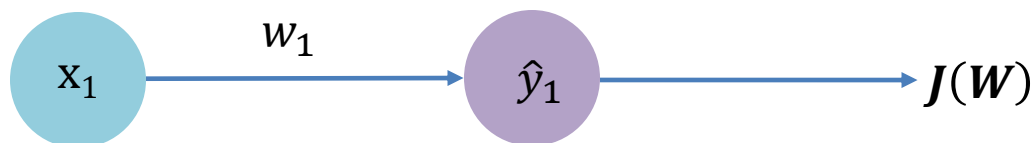


神经元群



神经元网络

神经元最简单的例子



随机参数

随机输出

神经网络实例

Weight = 0.5

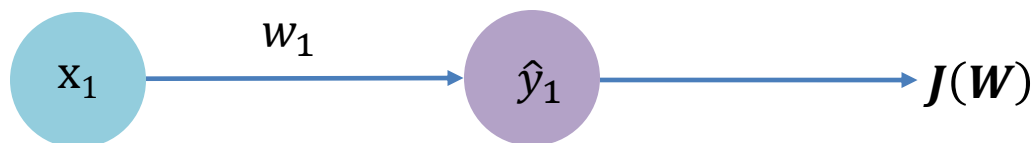
`Def neural_network(input_data, weight):` `prediction = input * weight` `return prediction`

这个神经网络能干吗？

学习一个模式

我们如何去**学习模式**呢？学习的是**什么**？

我们如何去**学习模式**呢？学习的是**什么**？



学习模型参数

损失函数（误差函数）

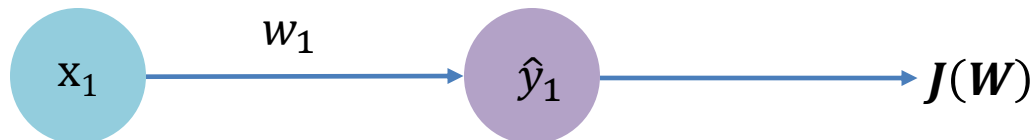
```
Def compute_error(input_data, weights, goal_prediction):  
    prediction = neural_network(input_data, weights)  
    raw_error = prediction - goal_prediction  
    normalized_error = raw_error**2  
    return normalized_error
```

学习目标：
损失值变为0

学习方法：
改变权重使损失为0

我们如何去找**正确的参数**呢？

我们如何去找**正确的参数**呢？



损失函数（误差函数）

```
Error = compute_error(input_data, weights, goal_prediction)
```

我们实际上只关心Error和Weights的变化
只要最终Error能变成0即可

最简单的方法



依次尝试



选择正确的变化

我们如何去找**正确的误差变化**呢？

我们如何去找**正确的误差变化**呢？（冷热法）

```
weight = 0.5
```

```
input = 0.5
```

```
goal_prediction = 0.8
```

设定一个初始误差变动幅度

```
step_amount = 0.01
```

设定循环次数以达到降低损失

```
For iteration in range(1101):
```

```
    prediction = input * weight
```

```
    error = (prediction - goal_prediction)**2
```

判断损失值是否变小

```
    up_prediction = input * (weight + step_amount)
```

```
    up_error = (goal_prediction - up_prediction)**2
```

```
    down_prediction = input * (weight - step_amount)
```

```
    down_error = (goal_prediction - down_prediction)**2
```

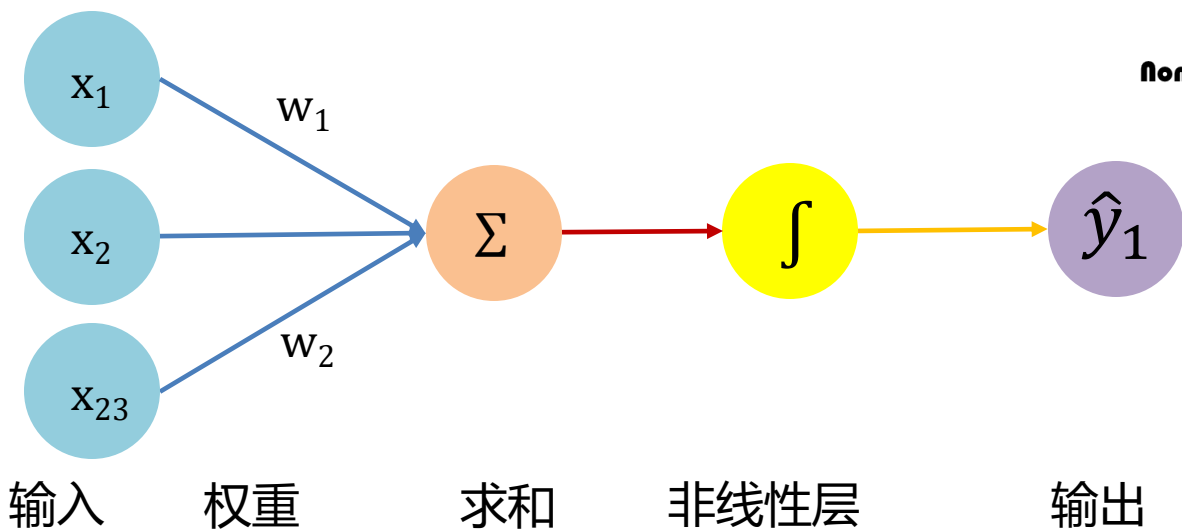
选择朝小的方向变动

```
    if (down_error < up_error): weight = weight - step_amount
```

```
    if (down_error > up_error): weight = weight + step_amount
```

在一个简单的神经网络中，我们需要考虑的内容较少，那么这些简单模式如何推导到更加复杂的神经网络中去呢？

神经元（单层感知机-较少输入）



linear combination of inputs

$$\hat{y} = g(w_0 + \sum_{i=1}^m x_i w_i)$$

Output

None-linear activation function

Bias

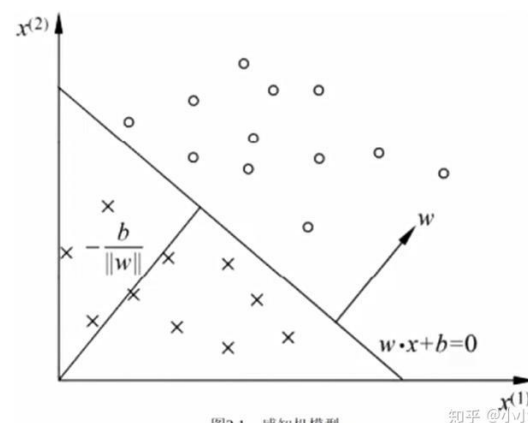
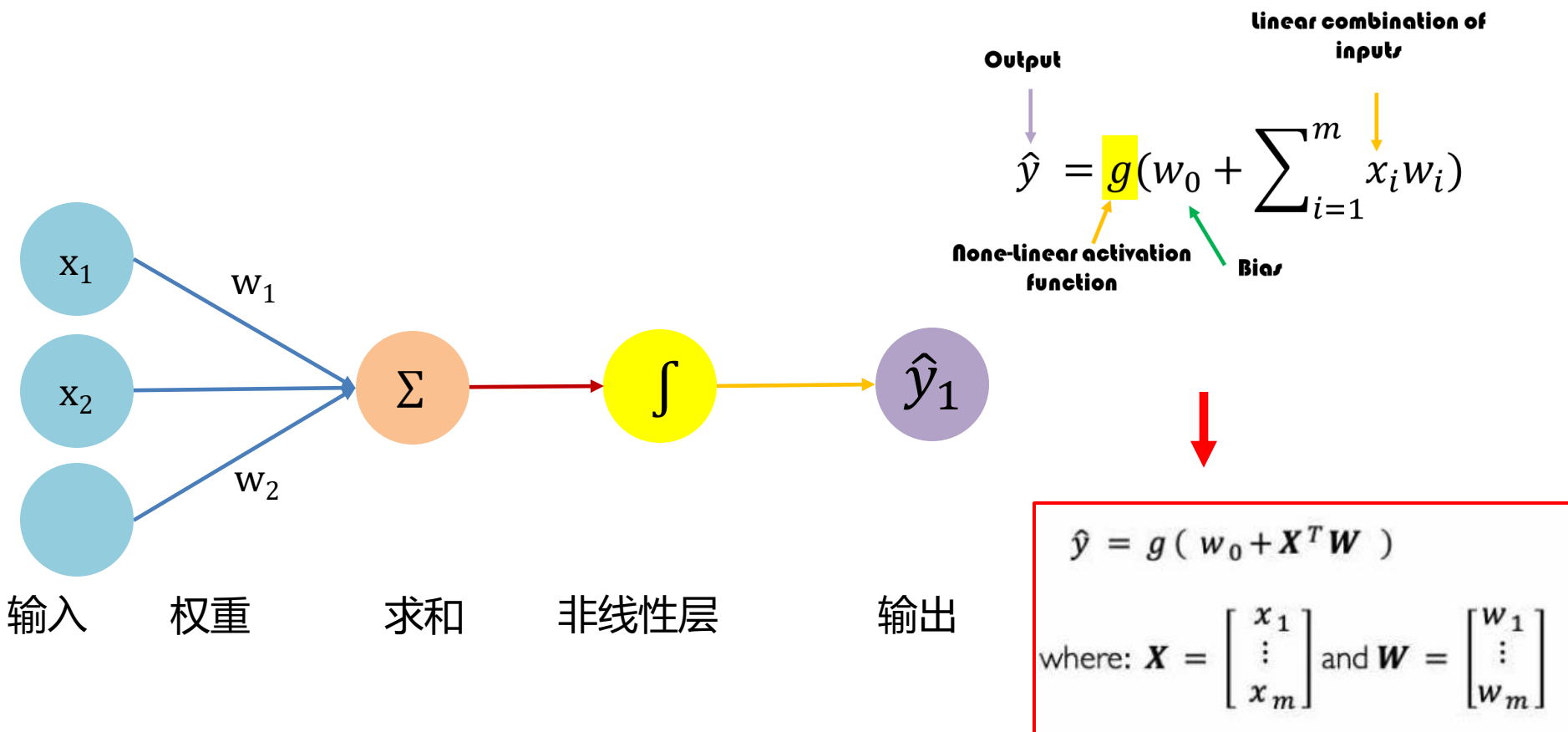


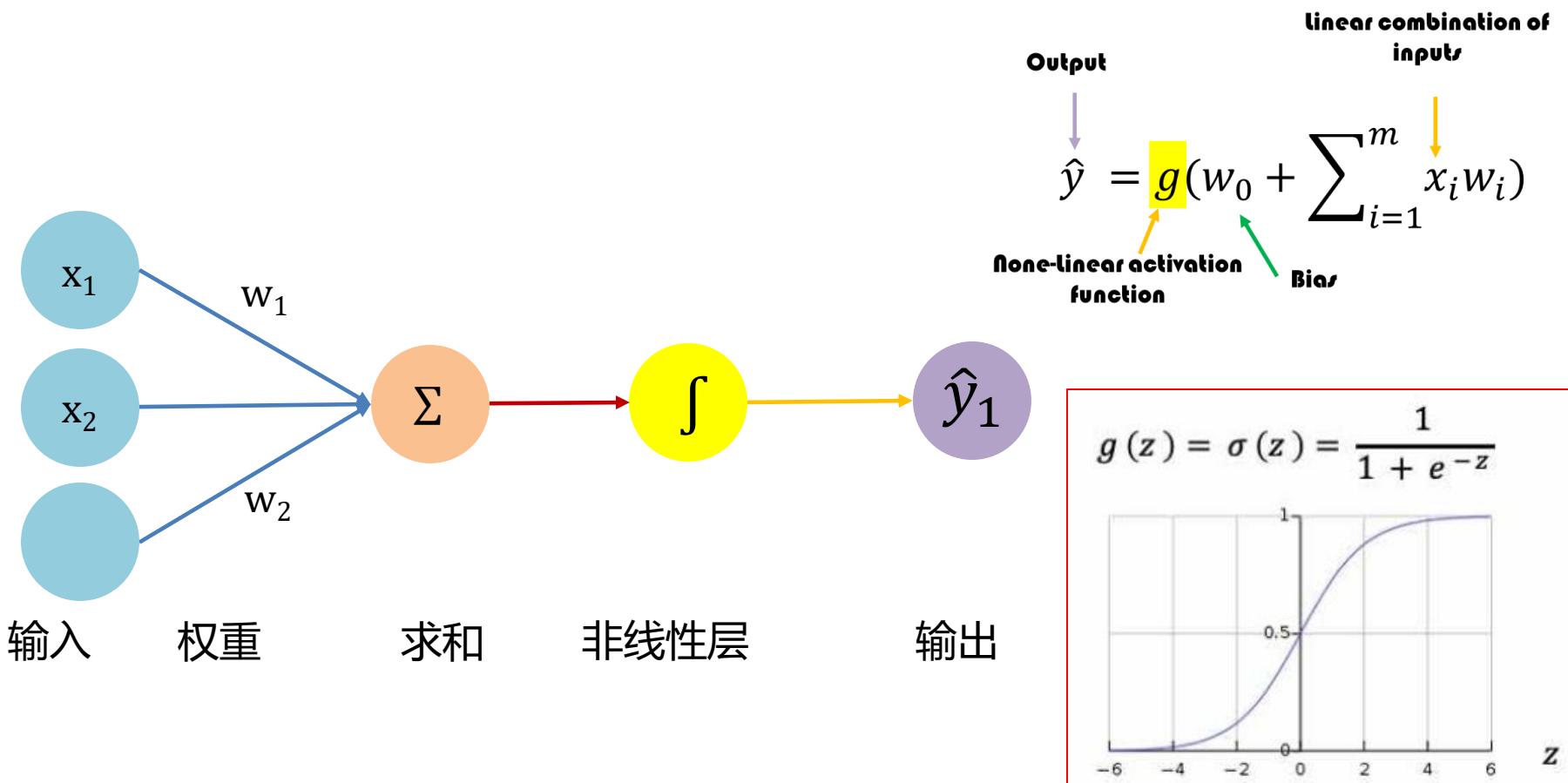
图2.1 感知机模型

知乎 @小小

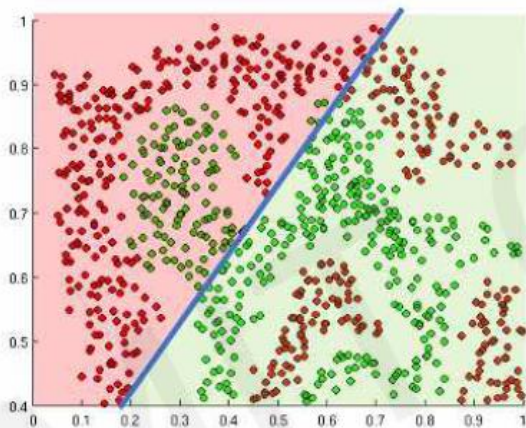
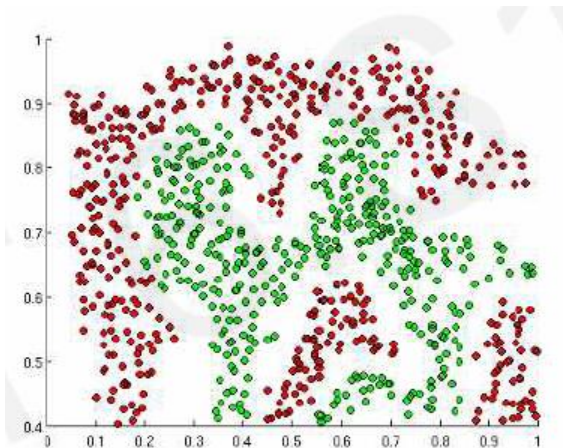
神经元（单层感知机-较多输入）



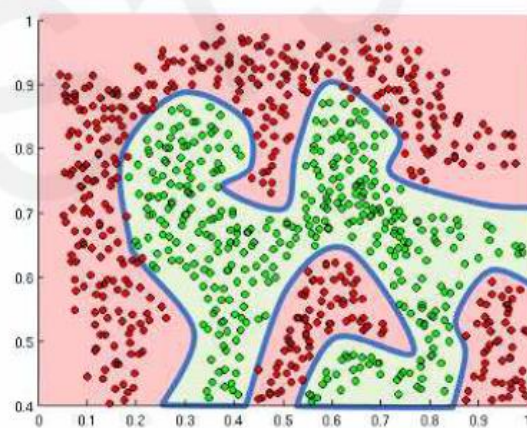
神经元（单层感知机-激活函数-1）



神经元（单层感知机-激活函数-2）



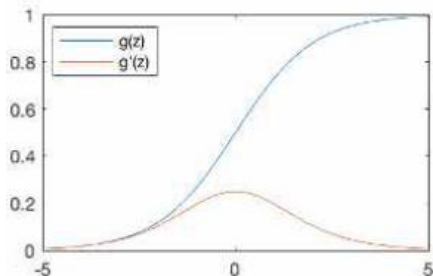
不使用激活函数的感知机分类



使用激活函数的感知机分类

神经元（单层感知机-激活函数-3）

Sigmoid Function

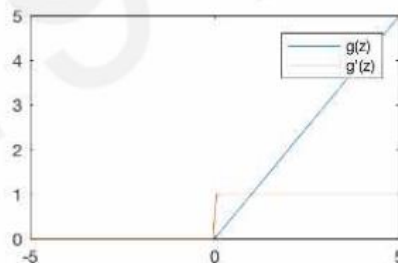


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

- 梯度消失
- 不是0均值
- 含有幂运算

Rectified Linear Unit (ReLU)

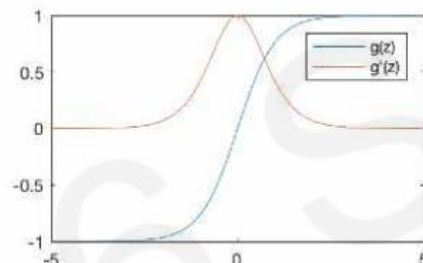


$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

- 解决梯度消失
- 收敛更快
- 不是0均值
- 部分没有输出

Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

- 梯度消失
- 含有幂运算

神经元运算举例（单层感知机单输出）

输入：

10

-20

4

-2

神经元权重：

0.8

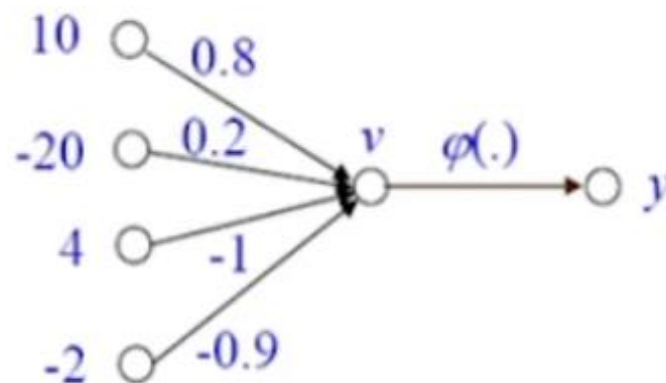
0.2

-1.0

-0.9

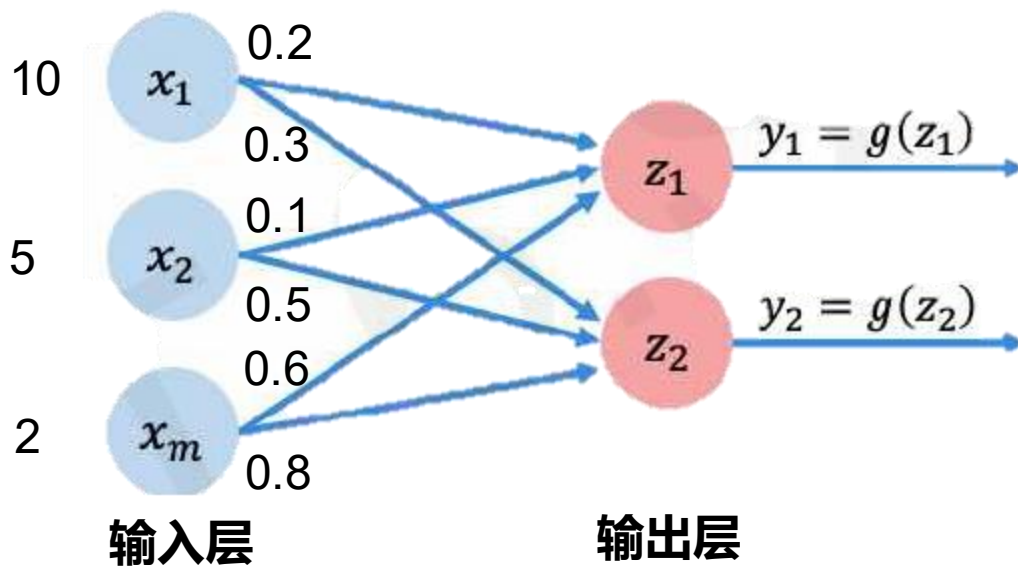
$$v_j = \sum_{i=1}^4 w_{ji} x_i$$

$$v_j = 1.8$$



$$y_j = g(v_j + 0.1) = \frac{1}{1 + e^{\{-(v_j + 0.1)\}}} = \frac{1}{1 + e^{\{-(1.8 + 0.1)\}}} = 0.8699$$

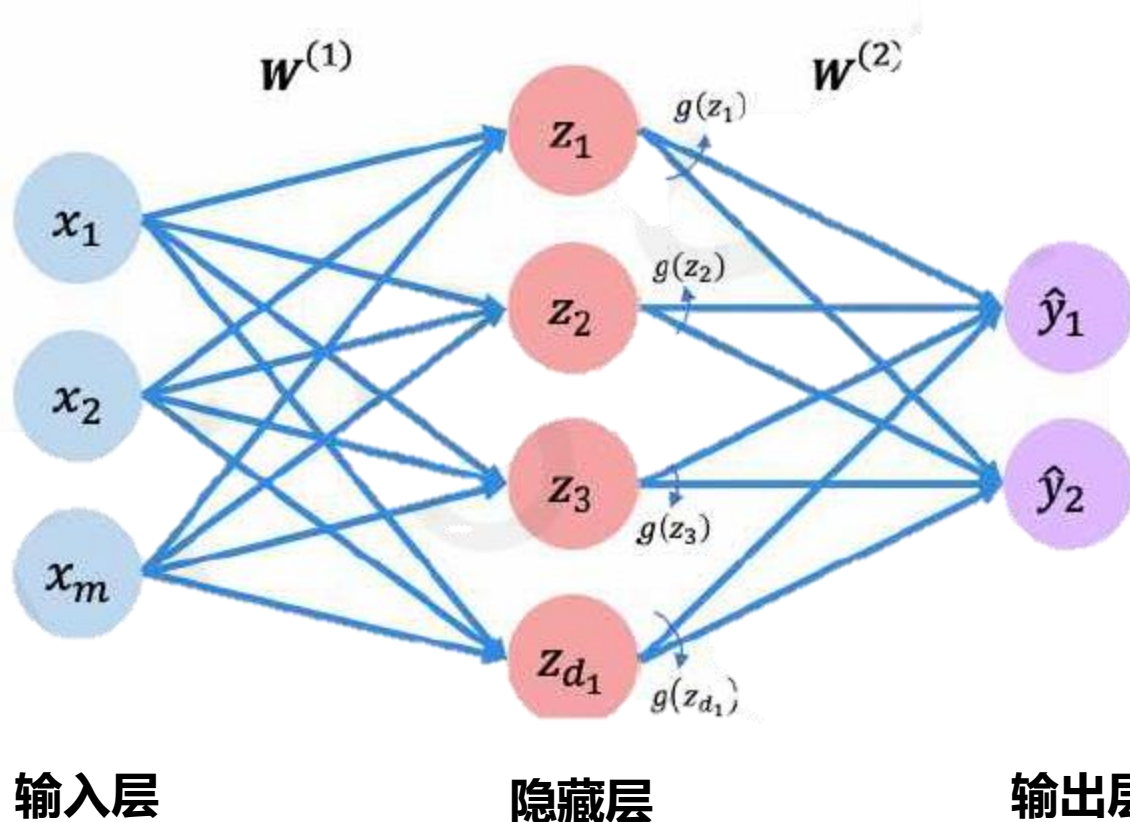
神经元运算举例（单层感知机多输出）



$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

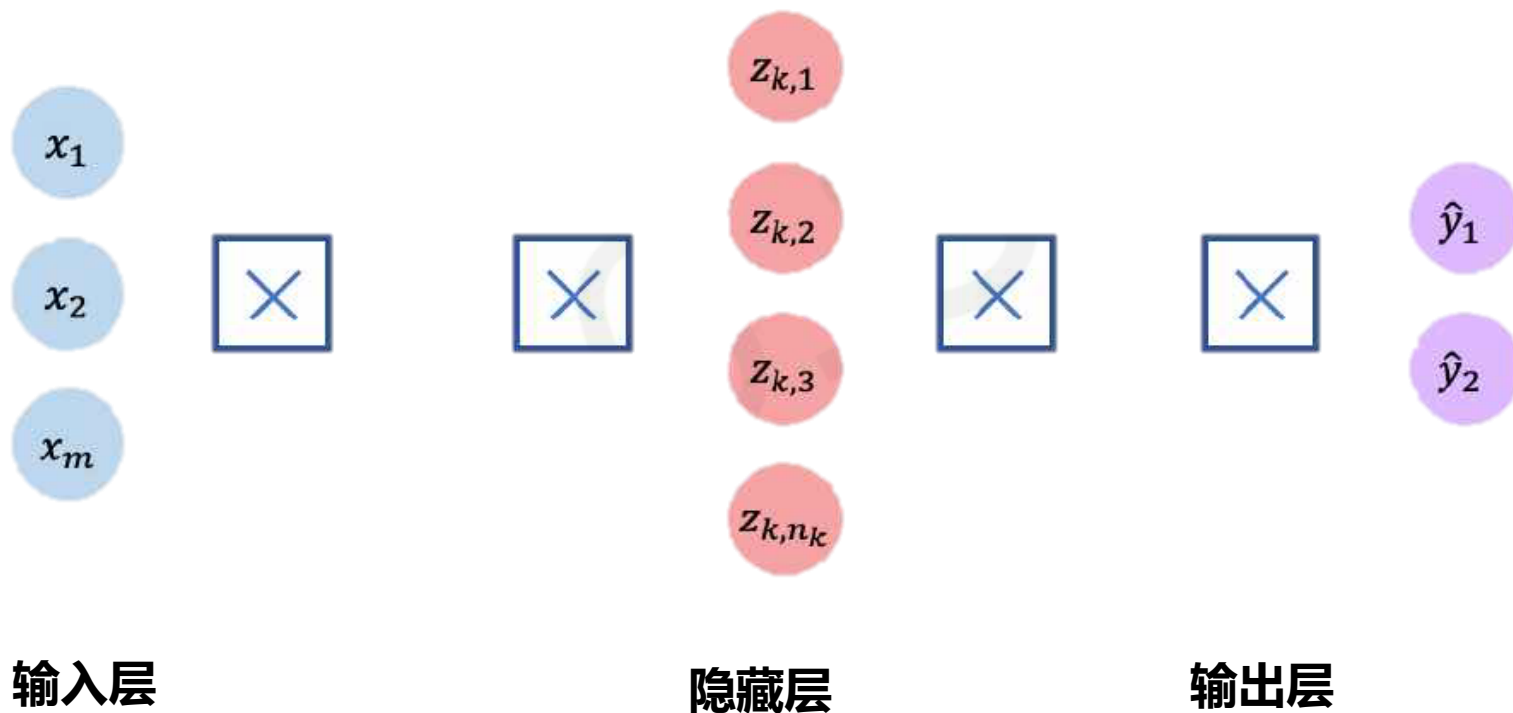
*此时，输入结点和输出结点全部连接，因此，这种又称为**全连接层**

神经元运算举例（单层神经网络）



$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

神经元运算举例（多层神经网络）



$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$



01

历史发展

02

知识基础

03

模型训练

04

相关思考

神经网络训练过程（简单样例）

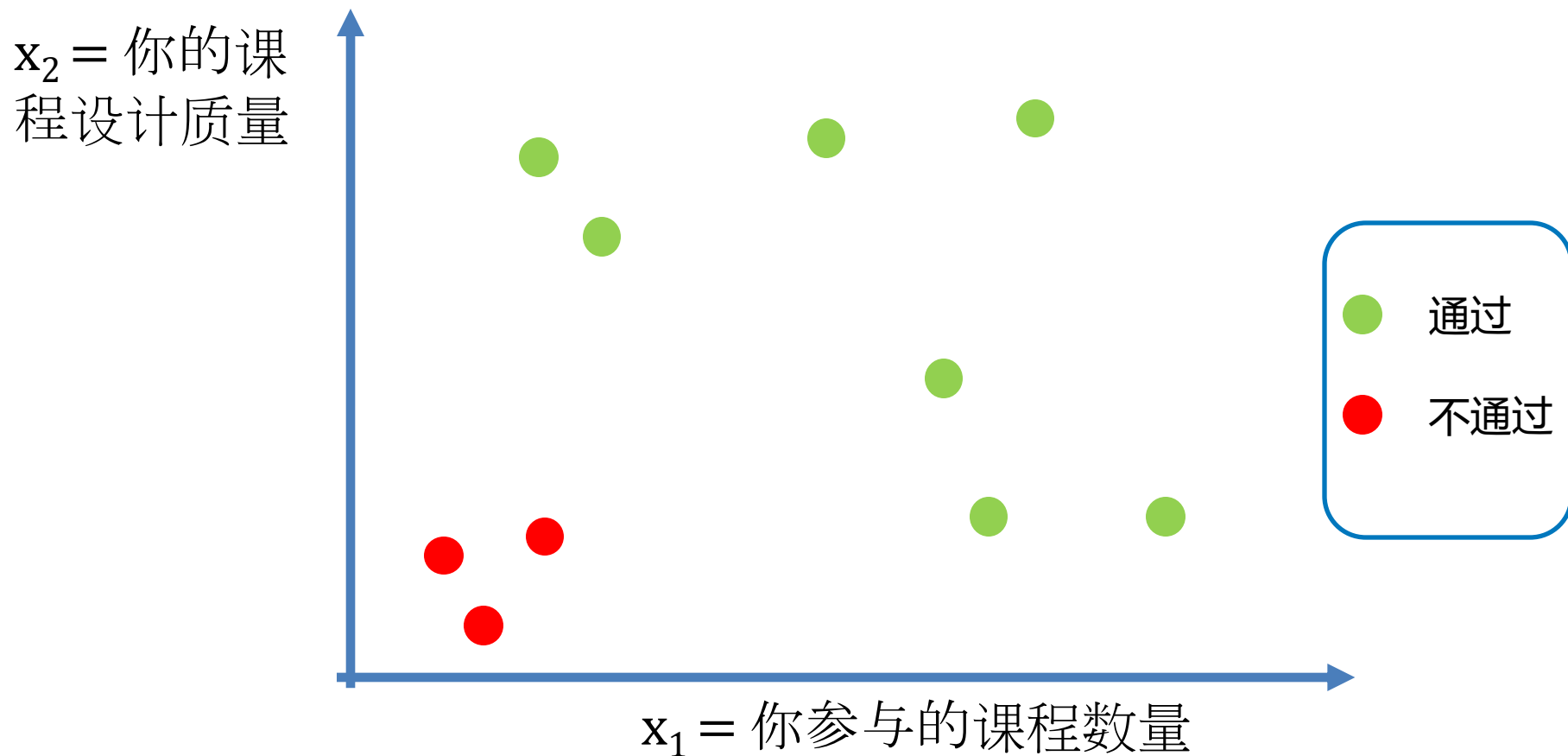
我能否通过这门课程？

我们设置一个简单的2维参数模型

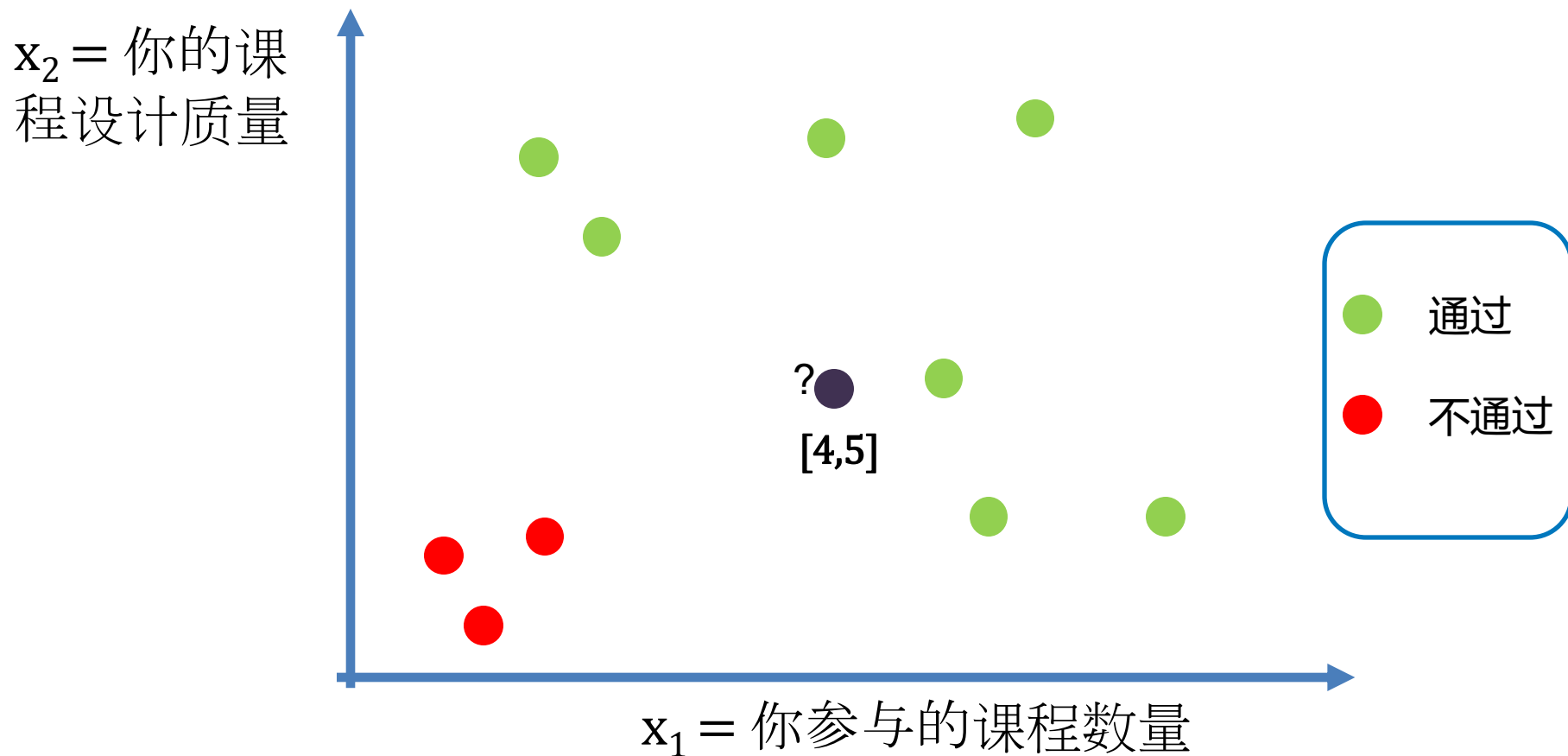
x_1 = 你参与的课程数量

x_2 = 你的课程设计质量

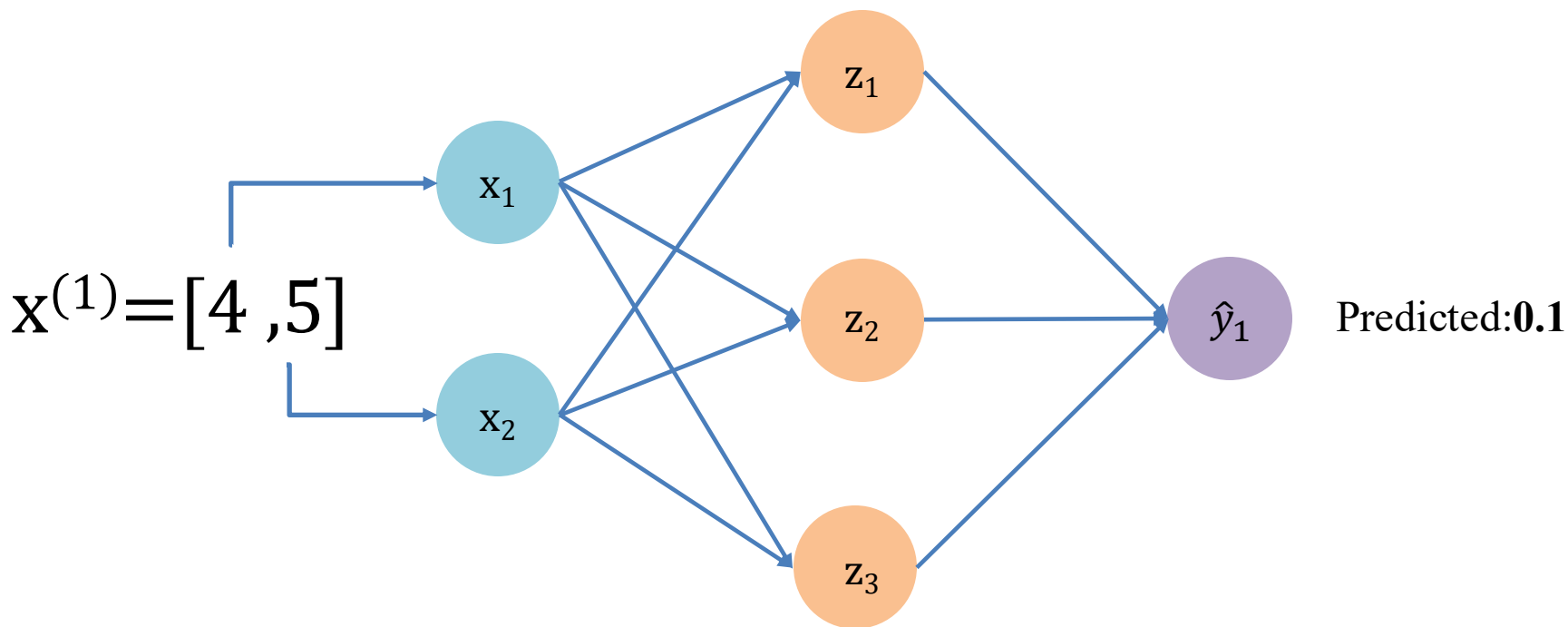
样例:我能否通过这门课程?



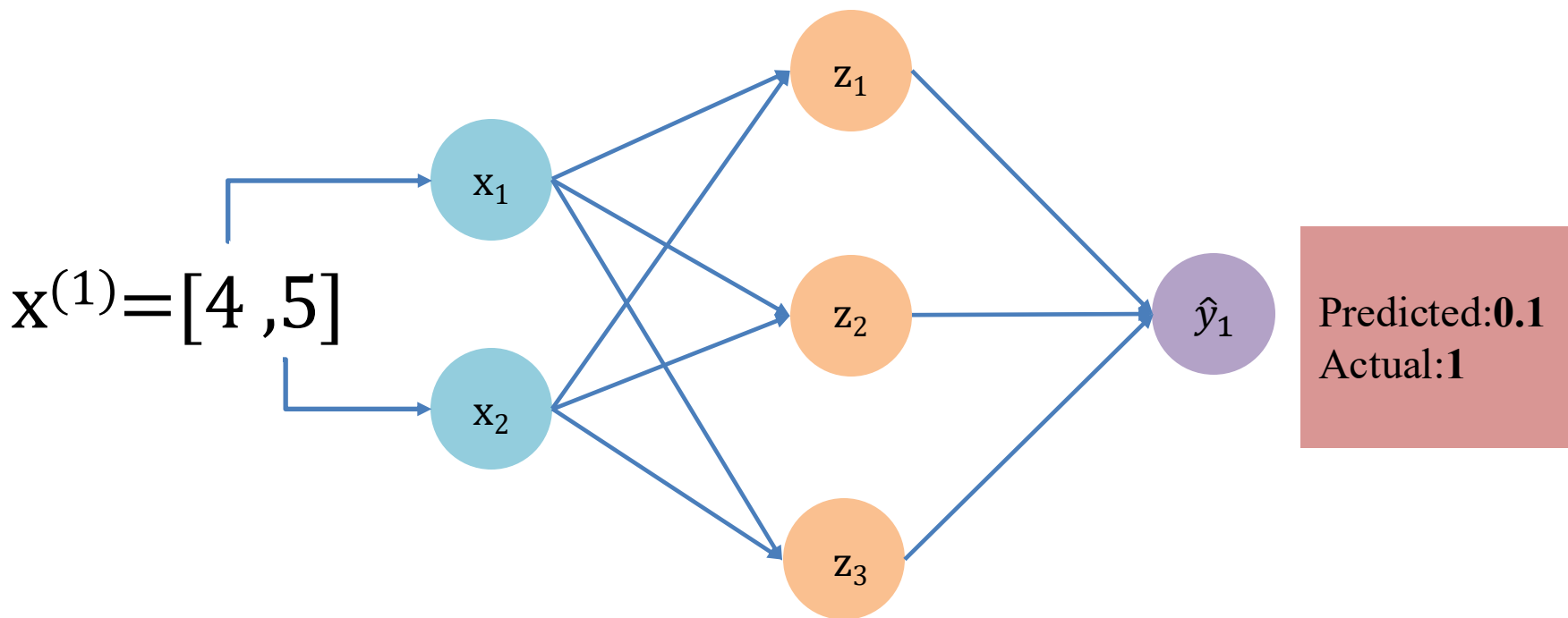
样例:我能否通过这门课程?



样例:我能否通过这门课程?

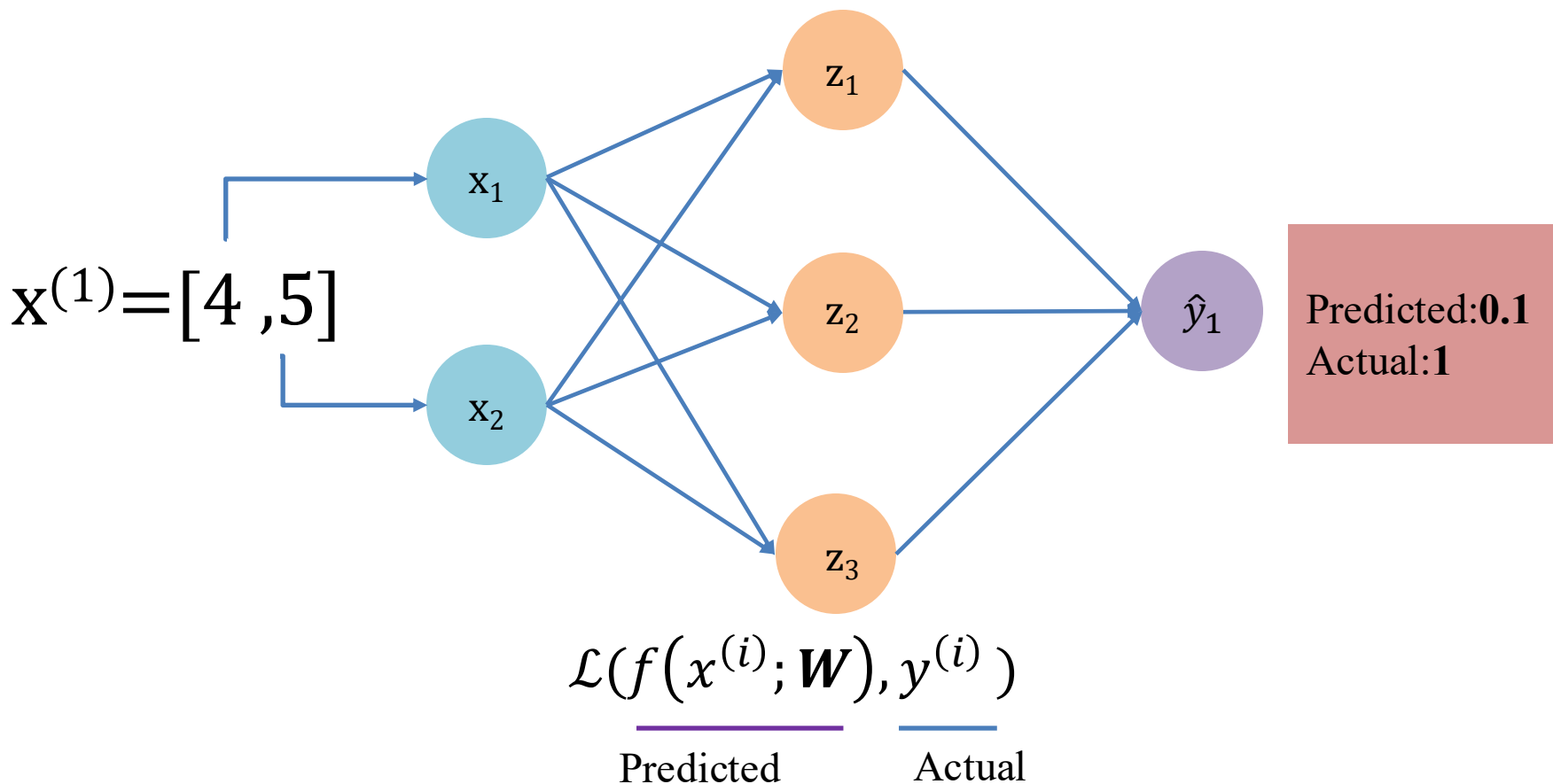


样例:我能否通过这门课程?



量化预测值的损失

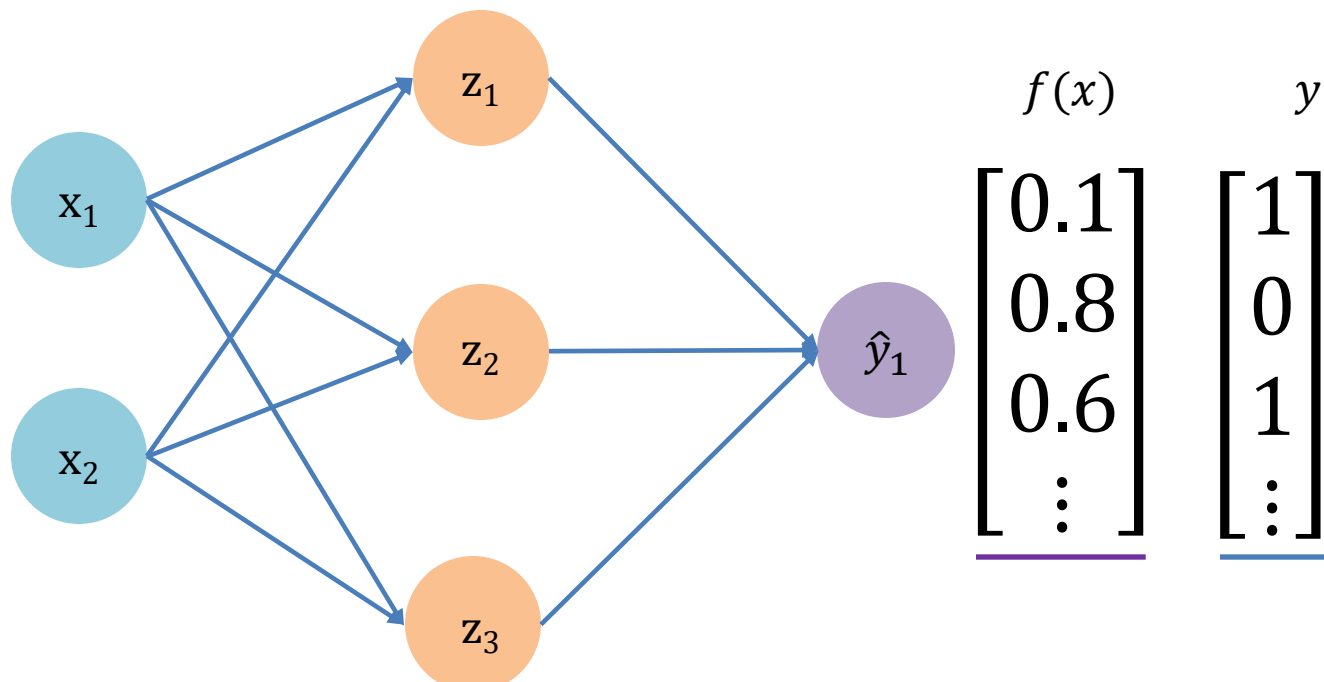
损失：衡量的是错误预测结果造成的损失情况



经验损失

衡量数据集上的所有损失情况

$$X^{(1)} = \begin{bmatrix} 4 & 5 \\ 2 & 1 \\ 5 & 8 \\ \vdots & \vdots \end{bmatrix}$$



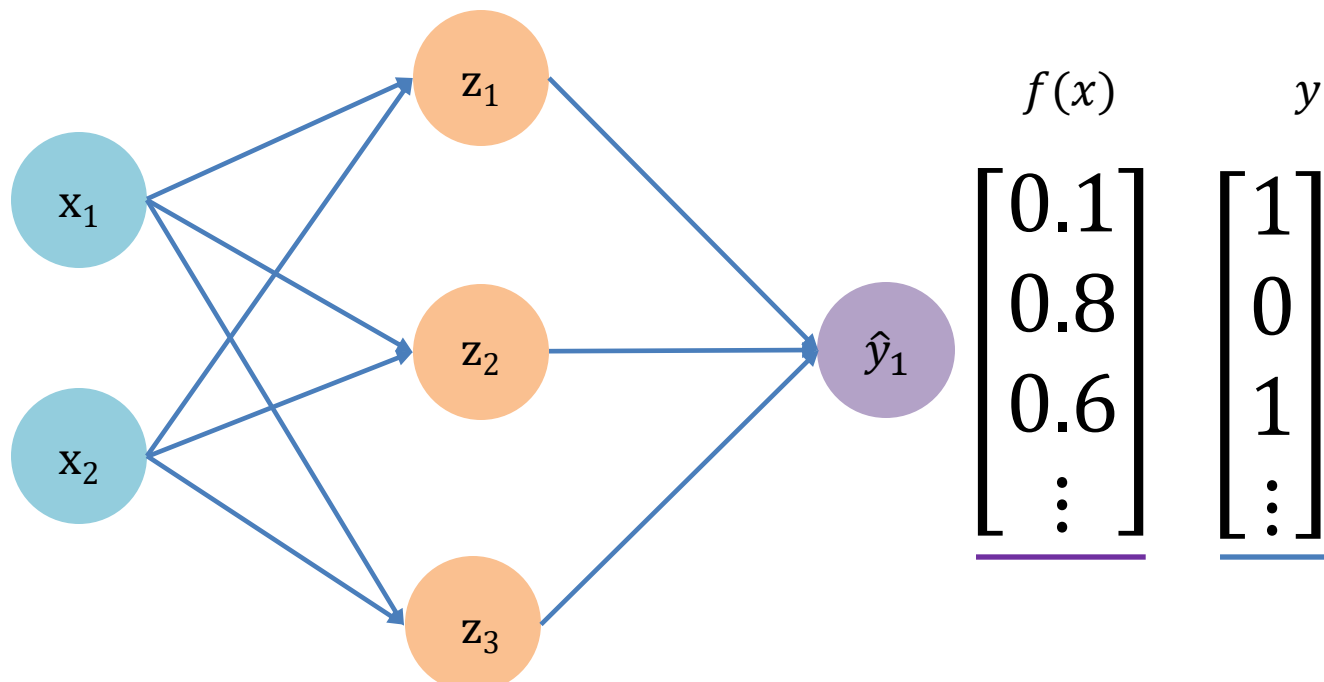
- 目标函数
- 开销函数
- 经验风险

$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{\mathcal{L}(f(x^{(i)}; W))}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}}$$

二元交叉熵损失

衡量二分类中预测值与真实值之间的分布情况

$$\mathbf{X}^{(1)} = \begin{bmatrix} 4 & 5 \\ 2 & 1 \\ 5 & 8 \\ \vdots & \vdots \end{bmatrix}$$

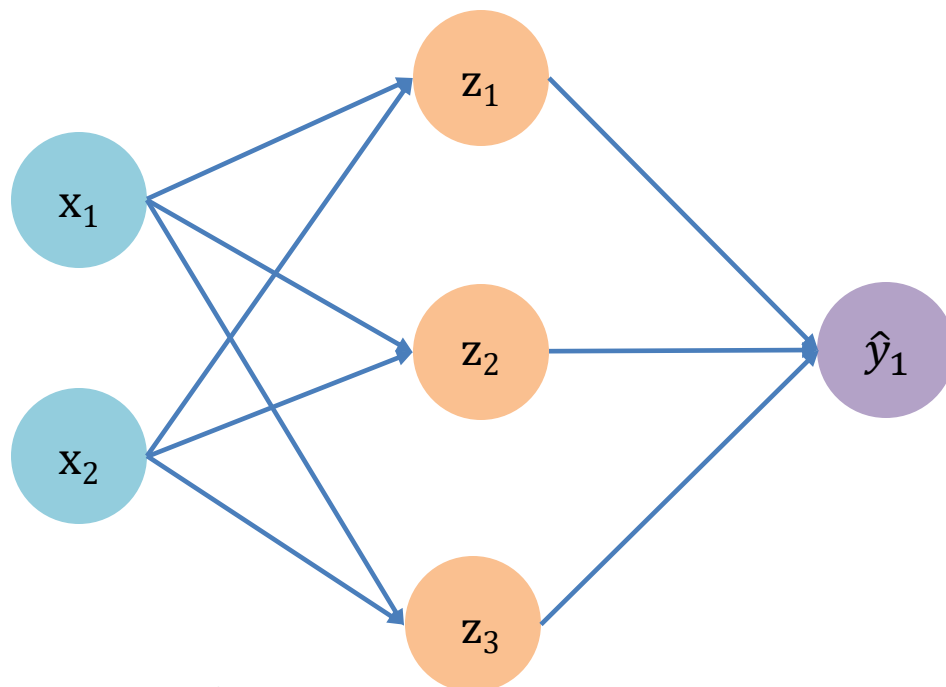


$$J(\mathbf{W}) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log(1 - \underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}})$$

均方误差损失

衡量的数据集上的所有损失情况

$$X^{(1)} = \begin{bmatrix} 4 & 5 \\ 2 & 1 \\ 5 & 8 \\ \vdots & \vdots \end{bmatrix}$$



$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 30 \\ 80 \\ 85 \\ \vdots \end{bmatrix} & \begin{bmatrix} 90 \\ 20 \\ 95 \\ \vdots \end{bmatrix} \end{array}$$

百分制成绩

$$J(W) = -\frac{1}{n} \sum_{i=1}^n \left(\underbrace{y^{(i)}}_{\text{Actual}} - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right)^2$$

Actual

Predicted

神经网络训练过程（训练网络）

损失优化

将损失降到最小以达到最好的预测效果

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$

$$W = \{W^{(0)}, W^{(1)}, \dots\}$$

当参数规模变大时如何高效寻找模型参数呢？

冷热法

```
if (down_error < up_error): weight = weight - step_amount  
if (down_error > up_error): weight = weight + step_amount
```

该方法很简单

难以确定参数变动方向和变动量

并不总是有效



损失函数（误差函数）

$$\text{Error} = ((\text{input} * \text{weight}) - \text{goal_pred})^2$$

我们实际上只关心Error和Weights的变化
只要最终Error能变成0即可

我们如何高效地使**Error**变成**0**呢？

03

模型训练

我们如何高效地使**Error**变成**0**呢?

损失函数 (误差函数)

$$\text{Error} = ((\text{input} * \text{weight}) - \text{goal_pred}) ** 2$$

Input是不变的

goal_pred也是不变的

损失函数可以变成:

$$\text{Error} = ((0.5 * \text{weight}) - 0.8) ** 2$$

画出函数图

Error

Weight随机初始点

Weight=0.5

Input=0.5

Error=0.3025

difference=-0.3025

Slope/斜率
确定方向

Optimal/
确定变动量

Gradient
descent

Weight目标

Weight = 1.6

Error=0

difference=0

最优点

Finding error minimums

weight

用简单梯度指导模型权重学习

```
weight = 0.5
goal_pred = 0.8
input = 0.5
for iteration in range(20):
    pred = input * weight
    error = (pred - goal_pred)**2
    delta = pred - goal_pred
    weight_delta = input * delta
    weight = weight - weight_delta
    print("Error:" + str(error) + "\t Prediction:" + str(pred))
```

引入简单斜率/梯度

用梯度指导权重改变

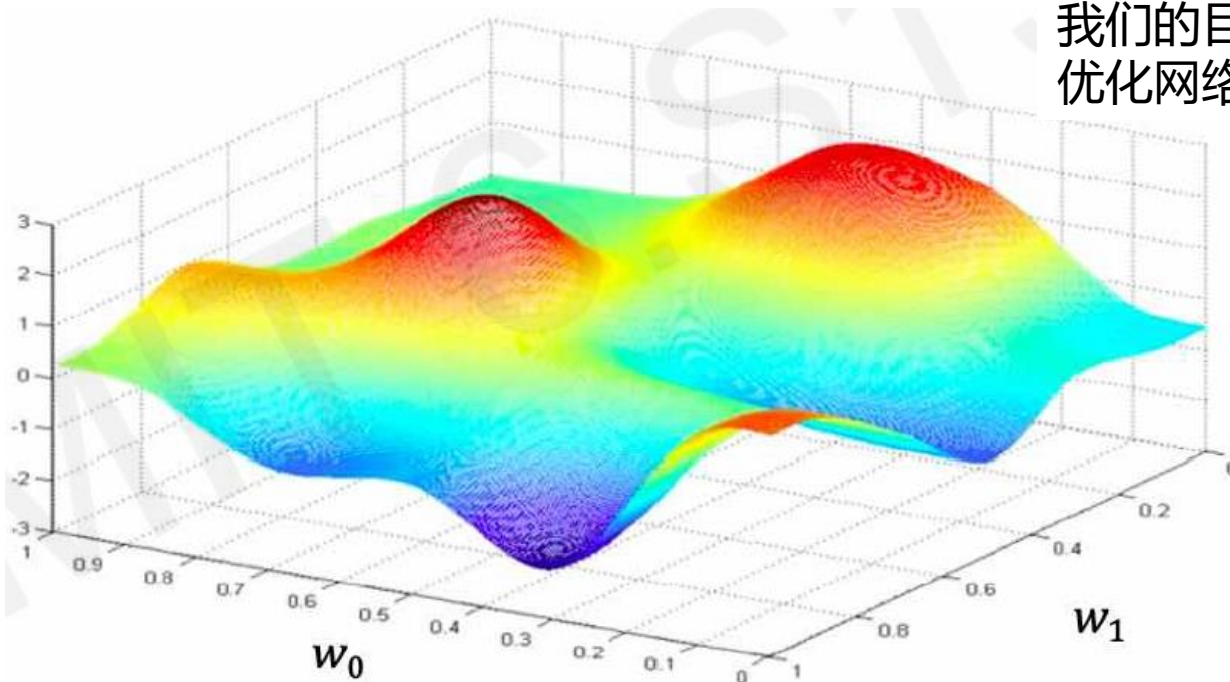
将上述基于梯度，如何将其推广到更大的模型学习中呢？

损失优化

$$W^* = \operatorname{argmin}_W J(W)$$

我们的目标是
优化网络参数

$J(w_0, w_1)$

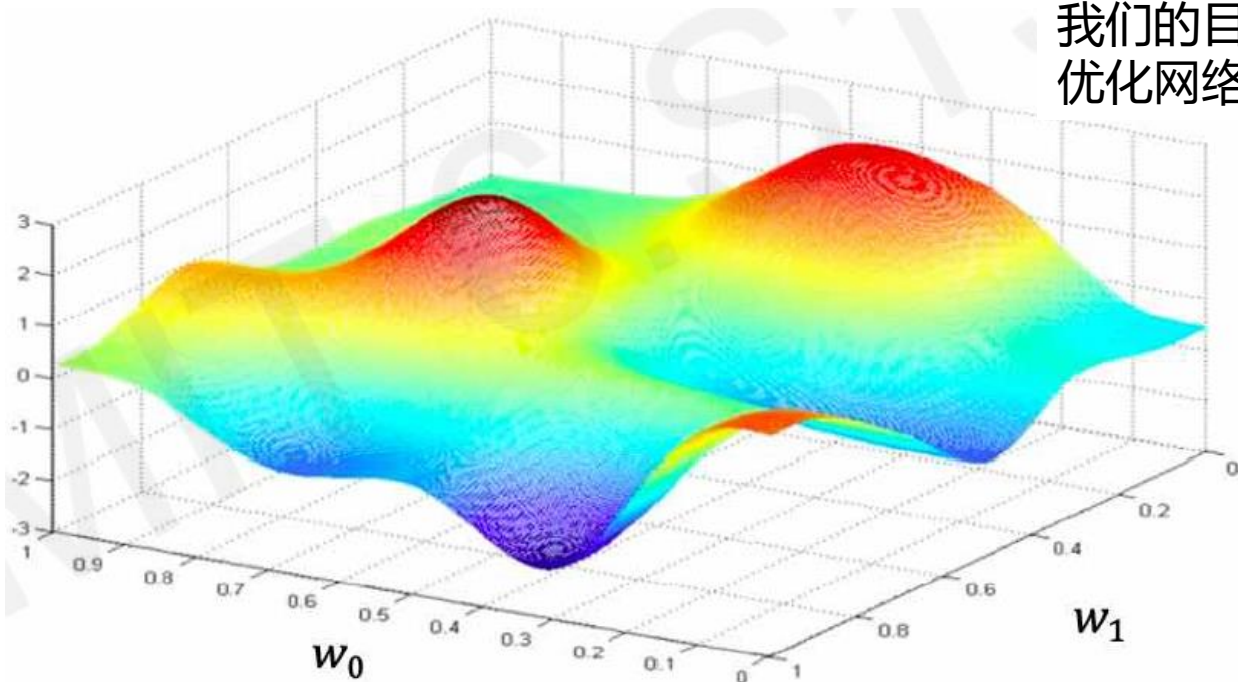


损失优化

随机选择一个初始的 (w_0, w_1)

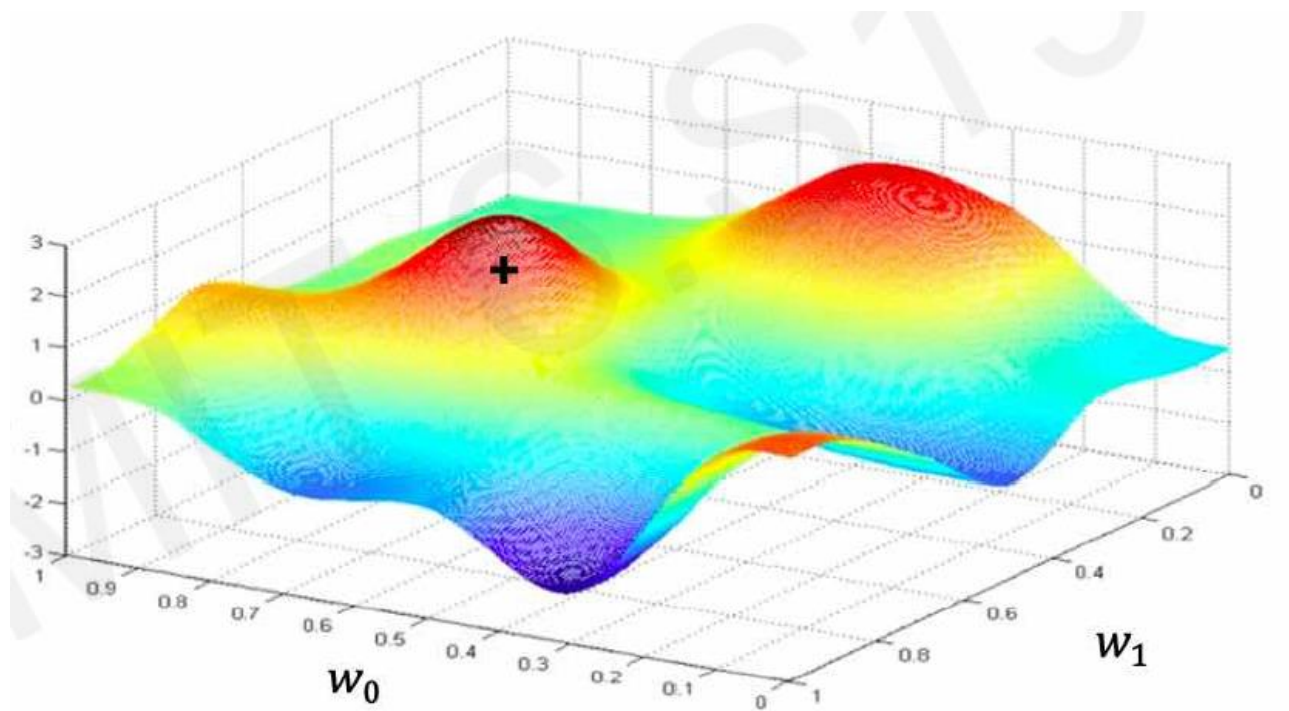
我们的目标是
优化网络参数

$J(w_0, w_1)$



损失优化

随机选择一个初始的 (w_0, w_1)

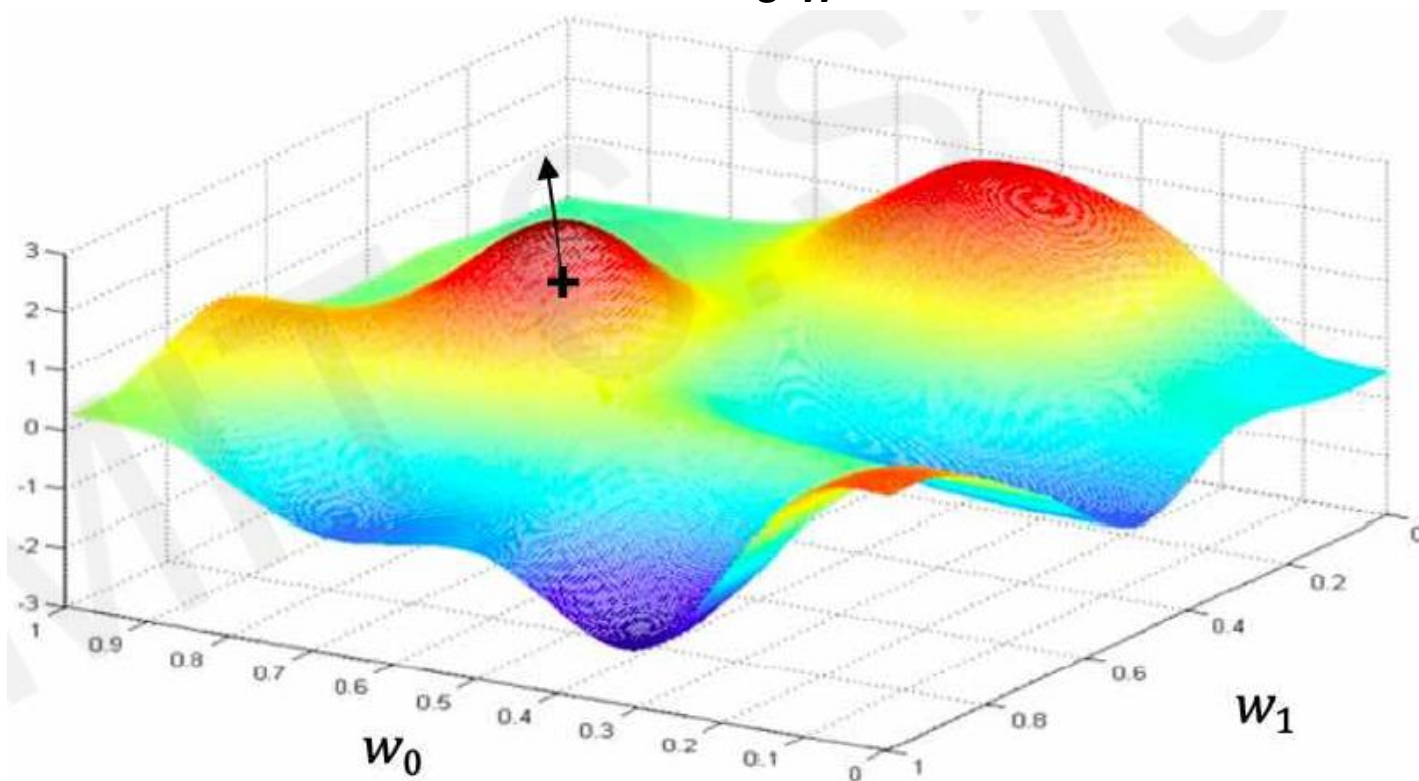


$J(w_0, w_1)$

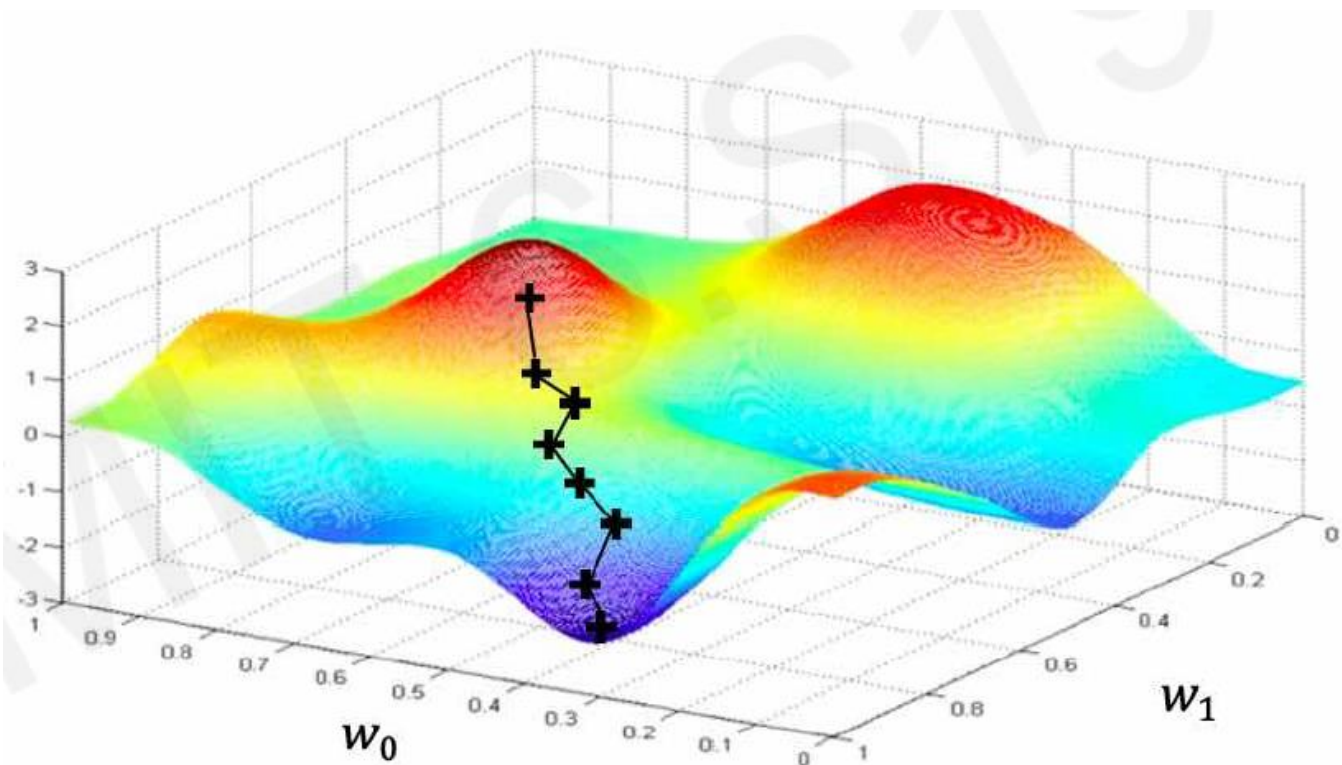
损失优化

计算偏导数 $\frac{\partial J(w)}{\partial w}$

$J(w_0, w_1)$



损失优化

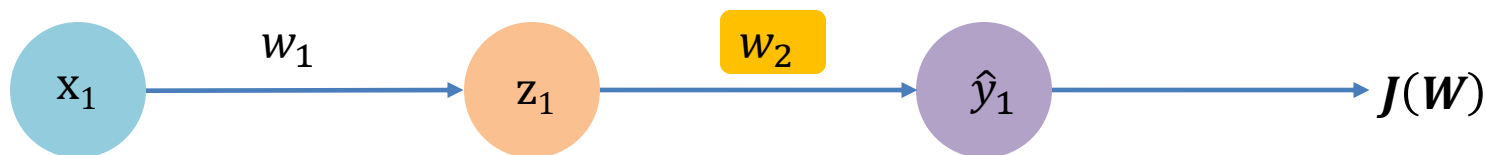
 $J(w_0, w_1)$

梯度下降

算法:

1. 随机初始化网络连接中的权重值 $\sim \mathcal{N}(0, \sigma^2)$
2. 循环如下步骤直到收敛:
3. 计算梯度 $\frac{\partial J(W)}{\partial W}$
4. 更新权重值 $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. 返回权重值

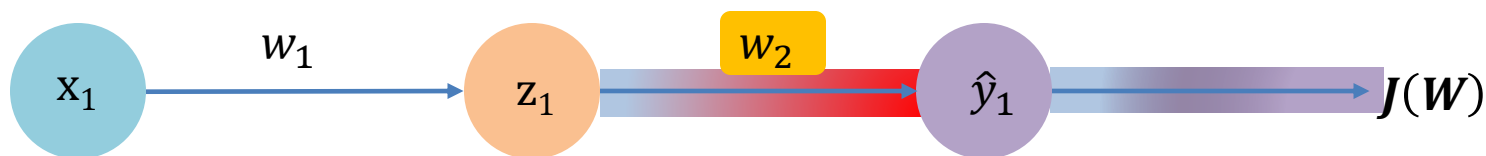
梯度下降：反向传播



为何改变权重值（比如 w_2 ）会影响最终的损失值 $J(w)$ ？

梯度下降：反向传播

使用链式求导法则，对 w_2 求导

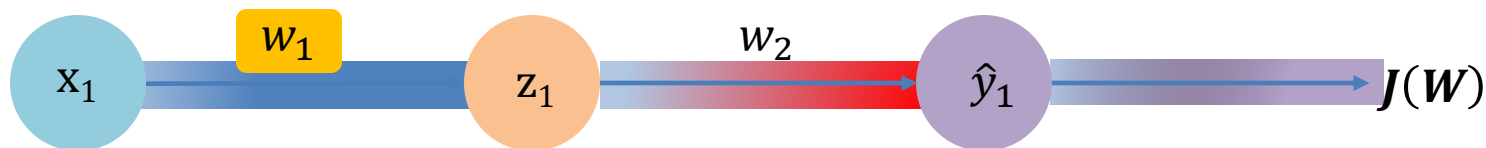


$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}_1} * \frac{\partial \hat{y}_1}{\partial w_2}$$

(The first term is underlined in purple, and the second term is underlined in red.)

梯度下降：反向传播

使用链式求导法则，对 w_1 求导



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}_1}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}_1}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

重复以上操作，直到网络中每层所有的连接权重均获得梯度值



01

历史发展

02

知识基础

03

模型训练

04

相关思考

1. 梯度下降的训练方法**难度大**么？
2. 梯度下降每次**下降幅度**怎么确定？
3. 如何确定我们**训练的程度**符合要求呢？
4. 我们每次拿**多少数据**来训练呢？



谢谢!